
JIT Simulation — Handoff Document

WHO YOU ARE BUILDING FOR

Team Axora. A Nigerian engineering team building JIT (Just In Time) — a retrofit Automatic Emergency Braking system for legacy commercial vehicles in Nigeria. This simulation is a competition demo for COUCH 2025. The builder is Emmanuel, solo developer, using Python + Streamlit in VSCode with GitHub Copilot and Cursor IDE.

WHAT THIS SIMULATION IS

A browser-based interactive simulation of JIT's decision logic. It is NOT a hardware simulation, NOT a real camera feed, NOT a vehicle dynamics model. It is a demonstration of the sensing → decision → actuation pipeline using simulated inputs. The goal is to make the logic visible to a non-technical competition judge.

WHAT IT IS NOT

- Not a real-time embedded system simulation
 - Not connected to any hardware
 - No actual camera or radar feed
 - No machine learning running inside it
 - The Kalman filter models sensor noise smoothing only — it is not an ML model
-

TECH STACK

- Language: Python 3.11+
- UI framework: Streamlit
- Charts: Plotly
- Kalman filter: filterpy (`filterpy.kalman.KalmanFilter`)
- Math/physics: numpy
- Auto-refresh: streamlit-autorefresh
- Camera: OpenCV placeholder only — button exists but is disabled
- IDE: VSCode + Cursor + GitHub Copilot
- Environment: Python virtual environment (venv)
- Version control: GitHub

FILE STRUCTURE

```
jit_simulation/
├── venv/                ← virtual environment, never touch
├── app.py               ← Streamlit entry point, UI only
├── jit_core.py          ← All simulation logic, no UI code
├── requirements.txt     ← All dependencies
└── README.md           ← Project description
```

STRICT SEPARATION RULE

`jit_core.py` contains zero Streamlit code. Zero UI code. Pure logic only. `app.py` contains zero physics or decision logic. UI only. It imports from `jit_core.py` and calls its functions.

Never break this separation. If you are tempted to put a calculation inside `app.py`, stop and put it in `jit_core.py` instead.

SIMULATION INPUTS

All inputs come from Streamlit sidebar sliders/toggles. No hardcoded values in logic.

Input	Type	Range	Default
Vehicle speed	Slider	40–120 km/h	80
Initial distance to obstacle	Slider	10–100 m	50
Obstacle closing speed	Slider	0–20 m/s	10
Road friction μ	Slider	0.2–0.8	0.6
Rain detected	Toggle	True/False	False
Camera confidence	Slider	0.0–1.0	0.8

SIMULATION OUTPUTS

Output	Type	Location
System state	Status card	Top of main panel
TTC actual	Status card	Top of main panel
TTC minimum	Status card	Top of main panel
Warning stage	Status card	Top of main panel
Distance over time	Plotly line chart	Main panel row 2 left

Output	Type	Location
TTC actual vs TTC min	Plotly line chart	Main panel row 2 right
Brake engagement (0–30%)	Progress bar	Main panel row 3 left
State timeline	Horizontal bar chart	Main panel row 3 right

CORE LOGIC — WHAT `jit_core.py` MUST CONTAIN

1. `KalmanTracker` class

- Uses `filterpy.kalman.KalmanFilter`
- State vector: `[distance, closing_velocity]`
- Input: noisy distance reading each timestep
- Output: smoothed distance + estimated closing velocity
- Purpose: models what the radar sensor output would look like after filtering

2. `calculate_ttc_actual(distance, closing_velocity)`

- Returns: `distance / closing_velocity`
- Guard: return infinity if `closing_velocity ≤ 0` (not closing)

3. `calculate_ttc_min(speed_ms, friction, t_actuation=0.3)`

- Formula: `speed_ms / (friction * 9.8) + t_actuation`
- `t_actuation` default: 0.3 seconds (300ms provisional — not yet measured on hardware)
- Returns: minimum seconds needed to stop safely

4. `FrictionEstimator` class

- Three layers in priority order:
 - Layer 1: IMU value if available (passed in directly)
 - Layer 2: If `rain=True` and no IMU reading, return 0.4
 - Layer 3: Default fallback $\mu = 0.5$
- In simulation, IMU layer is bypassed — friction comes from slider directly
- Class exists to mirror the real architecture even in simulation

5. `JITStateMachine` class

- Six states: `IDLE`, `THREAT_CANDIDATE`, `SUPPRESSED`, `ACTUATING`, `OVERRIDDEN`, `DEGRADED_CAMERA`
- Transitions:
 - `IDLE` → `THREAT_CANDIDATE`: when `TTC_actual ≤ TTC_min`
 - `THREAT_CANDIDATE` → `ACTUATING`: when camera confidence $\geq$ threshold (default 0.6)

- THREAT_CANDIDATE → SUPPRESSED: when camera confidence < threshold
 - SUPPRESSED → IDLE: after 1 second
 - ACTUATING → OVERRIDDEN: when driver_override = True (button press in UI)
 - Any state → DEGRADED_CAMERA: when camera confidence < 0.2 for 3+ consecutive seconds
 - DEGRADED_CAMERA brake rule: fire if $TTC_actual < TTC_min \times 0.6$ (radar-only fallback)
- State machine tracks: current state, time in state, state history for timeline chart

6. simulate_run(inputs) function

- Takes all slider inputs as a dict
- Runs the full simulation across N timesteps (default 100 steps)
- Returns: arrays of distance, TTC_actual, TTC_min, state_at_each_step, brake_engagement
- This is what app.py calls when the user hits RUN

UI LAYOUT — WHAT app.py MUST CONTAIN

Sidebar (st.sidebar):

- All sliders and toggles as described in inputs table above
- Disabled camera button with caption: "Feature coming in hardware phase"
- RUN button (primary)
- RESET button

Main panel:

- Row 1: Four `st.metric` cards — State, TTC Actual, TTC Min, Warning Stage
- Row 2: Two Plotly charts side by side using `st.columns(2)`
 - Left: Distance over time
 - Right: TTC actual vs TTC min — `TTC_min` as a horizontal red line, `TTC_actual` as a blue falling line
- Row 3: Two outputs side by side using `st.columns(2)`
 - Left: `st.progress()` for brake engagement (0.0 to 0.3 max)
 - Right: Plotly horizontal bar chart for state timeline

Color coding for state cards:

- IDLE: green
- THREAT_CANDIDATE: orange
- ACTUATING: red

- SUPPRESSED: grey
 - OVERRIDDEN: blue
 - DEGRADED_CAMERA: yellow
-

IMPORTANT CONSTRAINTS — READ THESE

1. `t_actuation = 0.3` is provisional. It is a placeholder. Do not hardcode it anywhere except as a named constant at the top of `jit_core.py`. When the hardware team measures it, one line changes.
 2. The camera confidence slider is a simulation proxy. In the real system this comes from an object detection model. Do not build any CV logic into this simulation.
 3. The friction slider is a simulation proxy. In the real system this comes from the IMU + rain sensor three-layer estimator. The `FrictionEstimator` class must exist in the code even though the simulation bypasses Layer 1 — it mirrors the real architecture.
 4. The simulation runs on button press, not continuously. It is not a live loop. When the user hits RUN, `simulate_run()` executes once and returns all arrays, which are then plotted.
 5. Never use `st.experimental_rerun()` or infinite loops in `app.py`. The simulation is triggered once per button press.
 6. All units internally are SI: metres, metres per second, seconds. Convert km/h to m/s on input before passing to any logic function.
 7. The brake engagement output maxes at 0.3 (30% pedal travel). Never exceed this in the simulation output.
-

WHAT IS DELIBERATELY LEFT OUT

- No real camera feed (placeholder button only)
 - No hardware interface
 - No GPS or real vehicle speed
 - No actual radar data
 - No drAlva AI layer (future phase — do not build)
 - No user accounts or data persistence
 - No mobile layout optimisation
-

CONTEXT FOR SYSTEM DECISIONS

If you are confused about why a decision was made, the reasoning is:

- Radar replaced ultrasonic because ultrasonic fails in rain and darkness — common on Nigerian roads
- Camera is validation only, not primary sensing — prevents false braking

- Confidence gate exists because radar alone cannot classify objects
 - Degraded camera fallback exists because rain + darkness make camera unreliable precisely when roads are most dangerous
 - Upper 30% pedal zone is intentional — driver can always override by pressing normally
 - t_actuation is provisional — not yet measured on actual hardware
-

COMPETITION CONTEXT

This simulation will be demoed live to COUCH 2025 judges. The demo flow is:

1. Set high speed, low friction, low camera confidence
 2. Hit RUN
 3. Show TTC lines crossing on the chart
 4. Show state machine moving to ACTUATING
 5. Show brake engagement rising
 6. Explain degraded camera fallback firing despite low confidence
 7. Point to disabled camera button — explain hardware phase is next
-